CST1510 Programming for data communication and networks

Resit Coursework 2 - Python Software - worth 50%

Resit Coursework 2: Simple Online Store System

Scenario: Build a Simple Online Store System

In this resit coursework, you will **develop a simplified online store system** that allows users to browse available products, add them to a cart, and complete a purchase. Unlike the original Smart Vending Machine scenario, this system will be designed for **physical products**, such as books, electronics, or clothing.

This project will be developed in **phases**, following the step-by-step instructions. The final version will include **Object-Oriented Programming (OOP), a client-server structure, a database for managing inventory and transactions, and an optional Graphical User Interface (GUI).**

Project Goals:

Your system should allow users to:

- **View Available Products:** List items with their names, prices, and stock availability.
- **Manage a Shopping Cart:** Add or remove items, update quantities, and calculate the total price.
- **Simulate a Checkout Process:** Confirm purchases, update inventory, and store transactions.
- **Use a Database for Inventory & Transactions:** Store product information and transaction history.
- **Implement Client-Server Communication:** The client handles the user interface, and the server manages inventory and order processing.

Submission Requirements

You must submit a **.zip** file labeled with your **name, surname, and student number** containing the following:

- **Python Code:** All `.py` files required to run the project (`client.py`, `server.py`, and any supporting modules).
- **Data Files:** If you use `.txt` or `.csv` files, include them. If using SQLite, include the `.db` file.
- **Video Presentation (5-7 minutes):** Instead of a written report, you must submit a video where you:
  - Explain your **implementation plan** and the **phases of development** (text-based functionality, OOP refactoring, client-server communication, database integration).
  - Demonstrate the **working software** by going through its functionality.
  - Discuss **challenges faced** and how you **solved them**.
  - Reflect on your **learning experience** and possible **improvements**.
- **Running Instructions:** A simple text file (`README.txt`) explaining how to run the program and any necessary dependencies.

Ensure that all required files are properly organized in the `.zip` submission for easy evaluation.

Implementation Phases by Steps

Step 1-2: Text-Based Application with Core Functions

Tasks:

Step a: Set up the product inventory

- Create a text file (`inventory.txt`) containing products in the format:
  ```
  ID,Product Name,Price,Stock
  1,Wireless Mouse,25.99,30
  2,USB Keyboard,49.99,20
  3,Laptop Stand,35.50,15
  ```
- Write a function `load_inventory()` to read the inventory file and load products into a list of dictionaries.

Step b: View available products

- Write a function `view_products()` to display the list of products.

Step c: Create a shopping cart system

- Create an empty dictionary `cart = {}` to store selected items.
- Implement `add_to_cart(product_id, quantity)` to:
  - Check stock availability.
  - Add the item to the cart if stock is sufficient.
  - Reduce stock in inventory.

Step d: View and manage the cart

- Implement `view_cart()` to display the cart's contents and total price.

Step e: Checkout process

- Implement `checkout()` to:
  - Show cart summary.
  - Confirm the purchase.
  - Update `inventory.txt` using `save_inventory()`.
  - Log transactions in `transactions.txt`.

Step 3: Refactor with Object-Oriented Programming (OOP)

Tasks:

Step a: Create a `Product` class

- Define attributes: `id`, `name`, `price`, and `stock`.
- Implement `update_stock(quantity)` to adjust stock.

Step b: Create a `Cart` class

- Define an attribute `items = {}` to store selected products.
- Implement methods:
  - `add_item(product, quantity)` – Add products to the cart.
  - `view_items()` – Display cart contents and total cost.
  - `clear_cart()` – Empty the cart after checkout.

Step c: Create a `Store` class

- Manage inventory and checkout process.
- Refactor previous functions into methods inside `Store`.

Step 4-5: Convert to Client-Server Model with Socket Communication

Tasks:

Step a: Set up the Server (`server.py`)

- Load inventory from `inventory.txt`.
- Implement functions:
  - `load_inventory()` – Load product data.
  - `get_product_list()` – Send available products to the client.

- `process_checkout(cart)` – Verify stock, update inventory, and confirm purchases.

Step b: Set up the Client (`client.py`)

- Implement functions:
  - `display_products()` – Request product data from the server.
  - `add_to_cart(product_id, quantity)` – Add items to the cart and send details to the server.
  - `view_cart()` – Display cart contents.
  - `request_checkout()` – Send cart data to the server for processing.

Step c: Implement Client-Server Communication

- Use socket programming:
  - The **client** sends requests (view products, add to cart, checkout).
  - The **server** processes requests and responds.

Step 6-7: Implement SQLite Database on the Server

Tasks:

Step a: Set up an SQLite database (`store.db`)

- Create tables:
  ```
  CREATE TABLE products (id INTEGER PRIMARY KEY, name TEXT, price REAL, stock INTEGER);
  CREATE TABLE transactions (id INTEGER PRIMARY KEY, product_id INTEGER, quantity INTEGER, total_price REAL, date TEXT);
  ```
- Populate the `products` table with inventory data.

Step b: Modify server functions to use the database

- Update functions:
  - `load_inventory_from_db()` – Retrieve product data from the database.
  - `update_inventory_in_db(product_id, quantity)` – Update stock after a purchase.
  - `save_transaction_in_db(cart)` – Log transactions in the database.

Step c: Test the database integration

- Ensure products are loaded and transactions are stored correctly.

Step 8-9: Add GUI with Tkinter (Optional for Higher Marks)

Tasks:

Step a: Create a main product display window

- Show available products with an “Add to Cart” button.

Step b: Create a cart management window

- Display cart contents and allow quantity updates.

Step c: Implement a checkout window

- Show order summary and confirmation buttons.

Step d: Connect GUI buttons to functions

- Ensure user actions trigger appropriate client functions.

Marking Criteria

Criteria	Marks Range
Foundational Requirements: Basic Python code, text-based interface, inventory management.	0-45%
Intermediate Requirements: OOP implementation, client-server communication with sockets.	46-70%
Advanced Requirements: SQLite database integration, dynamic inventory updates.	71-90%
Exceptional Work (+10%): GUI implementation with Tkinter, real-time stock updates, analytics with Matplotlib.	91-100%

Final Notes About Submission

- Your `.zip` file must contain **Python code, data files, and the report**.
- Provide a **short 10-minute video** explaining your work.
- Ensure your code is **well-commented and structured**.